

Lahore University of Management Sciences
School of Science and Engineering, Department of Artificial Intelligence
AI651 – Deep Learning for Space, Time and Graphs
Fall 2026

Assignment 1

Date Assigned: Saturday, Sep. 12th, 2026

Date Due: Friday, Oct. 2nd, 2026 11:55pm

Instructions

- While discussion with your peers is strongly encouraged, complete the assignment independently.
 - In order to gain the maximum benefit, try to perform solo brainstorming before any consultations. The thought process behind searching for the correct approach all on your own will be invaluable.
 - **Late Submission Policy.** Late homework solutions will be accepted with a 25% penalty till Saturday, Oct 3rd, 11:55 pm; and 50% penalty till Sunday, Oct 4th, 11:55 pm. No solution will be accepted after this.
 - The Professor and TAs reserve the right to hold a viva with any student if a discrepancy is found.
 - Students must submit one PDF report containing their answers, results, and findings through the LMS Assignments Tab.
 - For coding tasks, create a GitHub repository and provide its link on the first page of the report. Make sure that, at submission time, your repository is set to PUBLIC so the Professor and TAs can access it easily.
 - Students are welcome to use generative AI tools as partners in their learning journey. However, they are required to explicitly mention the prompt, its output, and edits made in the end of your report. For details, please refer to the AI policy listed on the course outline.
 - **Files.** Complete the three implementation cells in `Assignment1.ipynb`, run the notebook, and write the five assessed Task 1 responses in your report. Keep `harness/` and `requirements.txt` beside the notebook.
 - **Environment.** Use Python 3.11 or later; the notebook's Setup cell gives the install commands. Set `PA1_PRESET=smoke` while implementing and `PA1_PRESET=full` for the submitted evidence. CUDA is used when available; set `PA1_DEVICE=cpu` before import to force CPU.
-

Task 1 — Forecasting Across Heterogeneous Sensors

Consider a production line instrumented with vibration sensors, where the machinery does not always operate at the same pace. Your task is to build a forecasting service that uses a short window of recent vibration measurements from one machine location to predict its near future readings. The challenge is that the same location can exhibit different repeating patterns as operating conditions change, while a large slow-moving baseline can partially obscure the faster vibration cycle.

You will build the forecaster progressively, starting from fixed linear rules and moving toward models whose behavior adapts to the observed context. Along the way, you will test increasingly structured assumptions about the signal: first whether operating pace should explicitly determine the forecasting rule, then whether separating slow and repeating structure helps, and finally whether temporal relationships are better organized by position or by delay. The final comparison brings these ideas together and asks whether additional model flexibility is actually more valuable than the right inductive bias.

Each graded part identifies the mechanism to implement, the numbered notebook Outputs to inspect, and the matching report Response to write.

Part	What it adds	Outputs	Responses
0	the physical world, forecasting protocol, and supplied scaling	—	—
1	from one fitted linear map to Attention’s context-built mixing	1.1–1.3	1A–1B
2	centered moving-average decomposition	2.1–2.3	2
3	delay scoring and delay aggregation	3.1–3.2	3
4	compare four configurations and select deployment models	4.1–4.3	4

Central question
How much does the right inductive bias matter compared with a more flexible model?

0. The World and Forecasting Setup

The instrumented production line

Consider one production line with exactly four rotating production stations. Each station has one vibration accelerometer, sampled at 20 Hz (0.05 seconds per sample). The table connects each part of the simulated data to its physical interpretation.

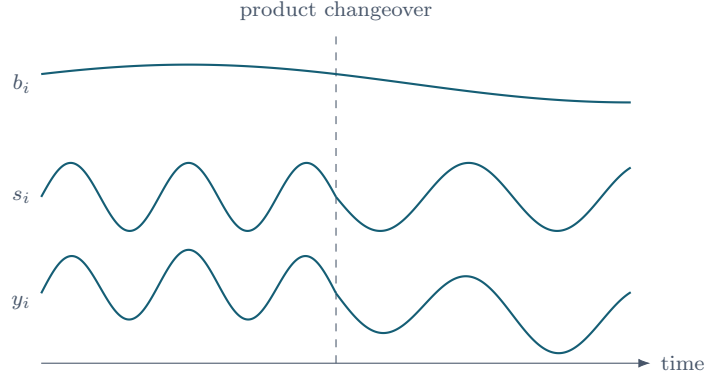
In the data	On the line
a repeating cycle	a dominant vibration pattern synchronized with the machine’s rotation (one cycle per revolution)
a change of regime	the line starts a new operating run. The product specifies a nominal speed range, while load, material, and controller conditions set one actual speed for that run; the whole line changes at once, so every station changes pace at the same moment
slow drift	slow load/control baseline variation plus sensor calibration drift

The observed signal

The dataset contains 15,360 samples (768 seconds) from each station. Station i 's reading in vibration acceleration (m/s^2) at time t is the sum of three components:

$$y_i(t) = \underbrace{b_i(t)}_{\text{slow part}} + \underbrace{s_i(t)}_{\text{repeating part}} + \underbrace{\epsilon_i(t)}_{\text{noise}}, \quad (1)$$

Here, $b_i(t)$ is the slowly varying baseline caused by load/control conditions and sensor calibration drift, $s_i(t)$ is one dominant rotating-vibration cycle, and $\epsilon_i(t)$ is small independent observation noise. Only their sum, $y_i(t)$, is observed by a forecaster. The supplied diagnostics may access the individual components, but a model may not.



The slow part changes smoothly across a context, while the repeating part changes spacing at the changeover. Only their sum, plus noise, is observed.

Global boundaries divide time into operating *runs*. Every 192 samples (9.6 seconds), the line switches to the next product in the repeating schedule Product A, Product B, Product C. Each product has a characteristic operating-speed range, represented here by its vibration period: Product A is 14–18 samples, Product B is 21–27, and Product C is 28–36, corresponding to cycle durations of 0.70–1.80 seconds. At the start of each run, load, material, and controller conditions determine one actual period within that product's range, which is shared by all four stations for the duration of the run.

Keep the following properties in mind:

- The same station experiences many operating paces, so station identity does *not* determine the appropriate forecasting rule.
- The stations differ modestly in vibration amplitude, mounting phase, calibration offset, and slow baseline drift. These are sensor and mounting differences, not different operating paces.
- The slow component varies on a 240-sample (12-second) time scale. Across a 96-sample context, it therefore appears as a smooth trend, while the faster 14–36-sample vibration remains visible. This motivates the moving-average decomposition in Part 2.

Forecasting task and information boundary

Let $L = 96$ be the context length and $H = 48$ the forecast horizon. At forecast origin t , a model observes the L most recent readings from one sensor,

$$\mathbf{y} = (y_i(t-L), \dots, y_i(t-1)) \in \mathbb{R}^L$$

and predicts its next H readings, $(y_i(t), \dots, y_i(t + H - 1)) \in \mathbb{R}^H$. A forecast origin is eligible only if its complete context and target lie inside one operating regime; no window may straddle a changeover. Every target must also lie completely inside its named temporal region:

Region	Target-time interval	Use
training	$[0, 9600)$	fit parameters and choose preprocessing
validation	$[9600, 12480)$	compare and choose deployment models
final test	$[12480, 15360)$	evaluate the selected models

Stations 1–3 are the *established stations*: only their training readings may fit models or preprocessing statistics. Station 4 is completely held out from fitting and scale selection, but still supplies its own L observed context at inference time. It is the unseen-station test case.

Anchoring and scaling

Supplied preprocessing

Each context is anchored by subtracting its final observed value, then divided by one fixed scale estimated from Stations 1–3 using training data only. This removes calibration offsets while preserving changes within the window, including slow drift. Predictions are transformed back to physical units before evaluation. Exact formulas are given in the appendix.

Leakage safety. Station 4 and the validation/test regions never contribute to the scale estimate.

1. Context-Dependent Forecasting

Learning goal

This section asks what must vary when different contexts require different continuations. You will move from one fixed linear rule to Attention.

Baseline: one shared linear map

We need a rule that maps a context of 96 anchored values to a forecast of 48 values. The simplest rule that uses the whole context is a matrix multiplication. We supply a fitted linear context-to-future map $\hat{\mathbf{z}} = \mathbf{x}^\top \hat{\mathbf{W}}$: a single matrix $\hat{\mathbf{W}} \in \mathbb{R}^{L \times H}$, fit once by least squares over all available training windows, with no optimizer and no iteration. We call it the **shared ridge** forecaster because the same fitted map is applied to every context from every sensor. The exact closed form is in the appendix (Equation (8)); the fact to carry forward is that $\hat{\mathbf{W}}$ is *one* fixed operator, chosen once and never adapted to the context it is currently facing.

Why the shared map must compromise

Consider one station across many Product A and Product C runs. Its actual period may be any value in 14–18 for the former and 28–36 for the latter, even though the station identity, amplitude, and drift may be similar. The next peaks therefore require different phase advances. For this continuum of paces,

continuation is not one fixed linear transformation of the observed samples: one fitted operator has no variable that selects the relevant phase advance, so least squares compromises across incompatible paces.

The forecasting rule therefore needs to depend on the current context. Two linear repairs clarify what that means.

Two linear repairs

Sensor-specific ridge fits one matrix per station and selects it using station identity. This does not solve the main problem: operating pace changes repeatedly within a station, including between runs of the same product, so each per-station matrix still faces the same compromise. It also cannot forecast for a station that never appeared in the fitting set.

Period-routed ridge conditions on the quantity that actually changes. It estimates the period of the current context, assigns that estimate to the nearest supplied 1.5-sample operating-pace band, and selects a matrix fitted from training windows in that band.

Period routing is the first forecaster here to receive temporal structure explicitly. A person has decided that period is the relevant quantity, chosen the estimator and band width, and routes an estimate to a hand-designed pace band.

Forecaster	What it assumes	How the forecast rule is chosen
Shared ridge	one context-to-future map fits every window	never chosen; the same fitted operator is applied
Sensor-specific ridge	windows differ mainly by which station produced them	by station identity, so unavailable for held-out station 4
Period-routed ridge	windows differ mainly by operating pace	by a period estimated from the context and matched to the nearest supplied pace band
Raw Attention	how positions should be compared can be computed from the window itself	constructed inside the model, per context, from learned projections

Attention: let the context construct the mixing rule

Period routing works only because a person named the right conditioning variable. Attention instead learns how to compare positions, without a hand-built period estimator or product router. Each position produces a *query*, *key*, and *value*; query–key comparisons produce a weight for every ordered pair of positions, and the output at each position is a weighted sum of values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (2)$$

where d is the model width, h is the number of Attention heads, and $d_h = d/h$ is the width of one head.

- W_Q, W_K, W_V are learned once, then frozen and reused at every position and for every context;
- the L^2 pair scores $QK^\top$ are *recomputed from the current window* every time;
- a new context therefore builds a new value-mixing operation without refitting or invoking a router, in contrast with the one fixed $\widehat{W}$ that shared ridge always applies.

Embed the context, apply two encoder blocks containing Attention and feed-forward sublayers with residual connections, flatten the result, and use a linear head to predict the horizon. We call this forecaster **Raw Attention**.

Raw Attention must learn around two avoidable difficulties:

- The slow component looks like a large smooth ramp across a 96-sample context, so the same representation must carry both drift and repetition. Part 2 addresses this problem.
- Attention scores every ordered position pair separately. The model must repeatedly discover that all pairs separated by the current context's 14–36-sample pace are informative for the same reason. Part 3 addresses this problem.

Part 4 tests whether either architectural change justifies its cost.

Outputs and responses

Notebook Outputs: 1.1 comparison; 1.2 forecasts; 1.3 Attention behavior.

Write: Response 1A and Response 1B.

Response 1A. Predict, measure, and revise the aggregate ordering.

Before running Output 1.1, record your expected ordering of the four forecasters for the established stations and for held-out Station 4. Then use Output 1.1 to:

- compare their performance on both station populations and across Products A, B, and C;
- quantify the key performance differences between the models;
- revise your expected ordering in light of the measurements; and
- explain the observed differences using the assumptions each model makes about the data.

Response 1B. Explain why the forecasting rule must depend on context.

Use Output 1.2 to explain why the single fitted matrix $\widehat{W}$ in Equation (8) must compromise when the same station operates at different paces. Identify that compromise in the forecast's phase or shape.

Then use Output 1.3 to explain how the same learned projections W_Q, W_K, W_V can produce different Attention weights for different contexts, and therefore different value-mixing operations. Contrast this with the fixed operator $\widehat{W}$ used by ridge.

Do not interpret Output 1.3 as evidence that Attention has recovered the physical period; it only shows that its mixing weights depend on the input context.

2. Separate Slow Structure from Repetition

Learning goal

This section introduces an architectural separation between slow structure and repetition. You will reason about a fixed filter before training a model, implement it, and measure whether the separation improves forecasting.

Why the raw representation is difficult

The slow baseline/calibration-drift component has a 240-sample time scale. Across a 96-sample context it therefore appears as a large smooth ramp rather than a cycle. This creates two problems:

- The encoder must represent the ramp while also capturing vibration that is much faster.
- A repetition score based on comparisons with shifted copies can be dominated by the ramp: a rising line remains similar to itself after many shifts. Part 3 will use exactly this kind of comparison.

Both problems suggest the same design: split the context into a slow stream and a repeating stream before the mixer sees it, then forecast each stream with a suitable component.

Centered moving-average decomposition

Use a centered moving average to split the anchored context $\mathbf{x}$ into a smooth component T and a remainder $S = \mathbf{x} - T$. For an odd width $m = 2q + 1$, interior positions use

$$T_t = \frac{1}{m} \sum_{u=-q}^q x_{t+u}, \quad (3)$$

where $q = (m - 1)/2$ is the half-width. The width m controls how much variation is absorbed into the smooth component.

Near the two boundaries, the averaging window would extend beyond the observed context. We therefore pad by repeating the nearest endpoint value. This keeps T and S the same length as $\mathbf{x}$ and preserves the exact reconstruction $S + T = \mathbf{x}$. Because these boundary windows contain repeated values, they behave slightly differently from interior windows; inspect this directly in the decomposition plot.

Choosing the width from evidence

The width m is the only design choice in Equation (3). Roughly, a wider averaging window follows slower structure, which is why the widths considered here are all much smaller than the 240-sample drift time scale but still comparable to or larger than the 14–36-sample vibration – so the choice is a genuine trade-off, not a knob with an obviously correct setting. Output 2.1 makes the resulting trend and residual visible for several candidate widths on the same context, so you can inspect how each one changes what ends up in T versus S before trusting any of them.

Do not treat S as the true repeating component. The generative component s_i in Equation (1) is not available to the model. The estimated remainder S is only whatever the moving average failed to absorb into T : it still contains the repeating vibration, together with any slow component the filter did not remove. Its quality depends on both m and the periods present, which is exactly why Output 2.2 measures that quality directly rather than assuming it.

How decomposition changes the model

Decomposition is an architectural change to the Raw Attention forecaster from Part 1, not a preprocessing step performed once. The decomposed forecaster

- sends S through the encoder;

- sends T through a linear trend head; and
- adds the two H -step forecasts.

The linear head handles the slow ramp, leaving the encoder to model repetition. Decomposition is also applied after every residual sublayer. Consequently, the next mixer receives a remainder stream rather than a stream in which the residual block has reintroduced slow structure.

This is the first switch in the final comparison. It changes the representation shown to the mixer but does not change Attention itself.

Outputs and response

Notebook Outputs: 2.1 filter/decomposition; 2.2 oracle period-recovery diagnostic; 2.3 decomposition ablation.

Write: Response 2.

Response 2. Complete `SeriesDecomposition.forward`. Then use Outputs 2.1–2.3 to:

- state the selected width and describe what changes visually in Output 2.1 and empirically in Outputs 2.2–2.3, including the validation comparison of Raw Attention and Attention plus decomposition;
- explain why a moving average is a sensible inductive bias for this drift-plus-repetition world, and why the training-only oracle diagnostic is a useful way to select its width here;
- give one situation where this bias would fail *or* name one alternative decomposition and the different assumption it would encode.

3. Mix by Temporal Delay

Learning goal

This section supplies the mixer with a second architectural assumption: useful temporal relationships are organized by displacement. Circular delay scoring is supplied; you will implement the aggregation that turns scored delays into a mixed sequence, and interpret why this organizing principle helps.

Why score delays instead of position pairs?

Attention scores every ordered pair of positions (t, s) independently. Pairs with the same displacement $t - s$ are not tied together and may receive different scores based on their content. For a repeating signal, however, displacement is the useful organizing principle. If one Product A operating run has an actual period near 16 samples, all pairs separated by about 16 samples are related for the same reason.

The delay mixer supplies this structure directly. It scores L temporal displacements instead of L^2 position pairs, then combines shifted copies of the sequence.

Circular delay scoring

A *delay* τ asks: how far back should the model look to find the context repeating itself? To score one delay, shift a copy of the sequence back by τ , multiply it by the original sequence position by position,

and sum the products. When the sequence repeats at that spacing, peaks align with peaks and the score is large; otherwise, positive and negative products tend to cancel.

All shifts are *circular*. An index that passes the beginning wraps to the end, and all index arithmetic is modulo L . Circular shifting makes every score a sum of the same L products, so the delays remain comparable. It also imposes an assumption: the final context sample is treated as adjacent to the first.

The layer does not correlate the raw context directly. It first projects the embedded context into queries q and keys k , so training can learn which features should align:

$$R(\tau) = \sum_{t=0}^{L-1} q_t k_{(t-\tau) \bmod L}, \quad \tau = 0, \dots, L-1, \quad (4)$$

where τ is a candidate delay.

Efficient delay scoring

Evaluating Equation (4) separately for all L delays would cost $O(L^2)$. The supplied `delay_scores` computes all delay scores together in $O(L \log L)$ using the FFT. You do not need to derive or implement this step; the important idea is that the model can compare many possible temporal delays efficiently.

Mixer design

The supplied mixer restricts attention to a fixed delay band, keeps only the top- K scoring delays, and uses a learned temperature to control how sharply weight is concentrated among them. The exact hyperparameters are listed in the appendix and are not part of the report task.

Output 3.2 provides a **signal-level recurrence diagnostic**. It compares circular correlation of the raw signal with that of its decomposed residual. Removing the slow component should make recurrence near the run’s period P and its multiples easier to see.

This diagnostic is *not* a plot of the trained mixer’s learned scores. Inside the model, Equation (4) is applied to learned query/key features; Output 3.2 only illustrates why organizing information by recurring delay can be a useful inductive bias.

Convert selected scores to weights

For the K selected delays $\tau_1, \dots, \tau_K$, convert their scores into normalized weights:

$$\alpha_j = \frac{\exp(R(\tau_j)/\vartheta)}{\sum_{r=1}^K \exp(R(\tau_r)/\vartheta)}, \quad j = 1, \dots, K, \quad (5)$$

where $\vartheta > 0$ is a learned temperature. The weights are positive and sum to one.

Delay aggregation

Each selected delay retrieves values from τ_j steps earlier. Combine them as

$$z_t = \sum_{j=1}^K \alpha_j v_{(t-\tau_j) \bmod L}, \quad t = 0, \dots, L-1. \quad (6)$$

Each example has its own selected delays and weights, shared across heads and features within that example. Using several delays allows the mixer to combine multiple plausible recurring patterns instead of committing to a single lag.

Combining several delays rather than committing to one is a hedge under squared error. When multiple spacings are plausible, the blend loses slightly if one candidate turns out to be correct but avoids the much larger error of committing confidently to the wrong candidate.

Outputs and response

Notebook Outputs: 3.1 supplied delay-score check; 3.2 signal-level recurrence diagnostic.

Write: Response 3.

Response 3. Implement `aggregate_delays` (Equation (6)); `delay_scores` is supplied, and Output 3.1 confirms it against a direct calculation on a small worked example. Verify the numerical values and indexing direction using the immediate aggregation worked example after your implementation. Then use Output 3.2's *signal-level* recurrence diagnostic to answer:

- Which lags show the strongest recurrence in the diagnostic, and how do they relate to that operating run's actual period P and its integer multiples?
- Why does decomposing the context first (removing the slow drift) make the delay evidence cleaner than in the raw signal, and why does that support the delay mixer's inductive bias?
- Delay mixing assumes that useful temporal relationships are organized by a small set of recurring displacements. Describe one situation where this assumption would be the wrong one to build in – for example, an aperiodic event, a period that drifts quickly within one context, or an isolated transient – and explain briefly why delay-based mixing would mishandle it.

Do not describe this figure as learned model scores: the model uses learned q/k delay scores, while the figure is a diagnostic correlation computed directly from the signal.

4. Compare Designs and Deploy

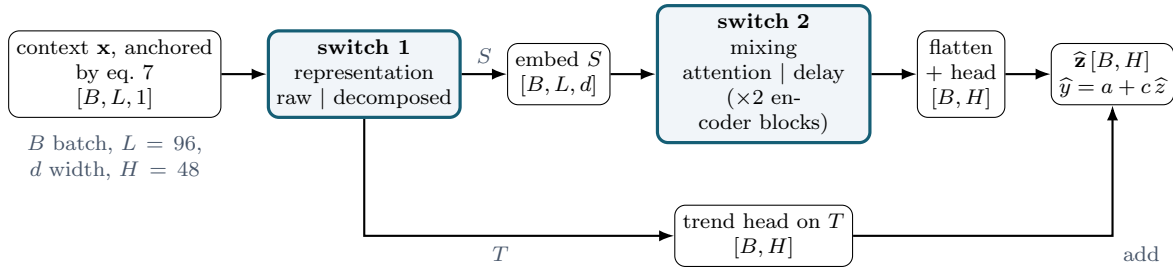
Learning goal

This section uses a 2×2 experiment to compare validation performance and cost, describe the observed relationship between the two switches in this benchmark run, and select deployment models. The comparison supports an empirical choice; it does not identify why an observed difference arose. Before running it, form a hypothesis: does each upgrade help on its own? If both help, is the combined model's gain roughly the sum of the two individual gains, larger than that sum, or smaller? Both upgrades address related aspects of the same drift-plus-repetition structure, so there is a concrete reason the two gains might overlap rather than stack – test whether the evidence in Output 2.3 and this section's four-model comparison supports that.

The two upgrades address different hypotheses:

- **Decomposition** changes the representation shown to the mixer because slow structure can crowd out the repeating component.
- **Delay mixing** changes the mixer because useful relationships may be organized by temporal displacement.

Either upgrade can be enabled independently. Replacing Attention with Equations (4)–(6) produces the **delay mixer**. Combining that mixer with the decomposition stream from Part 2 produces the **Autoformer-inspired** variant. All four variants use the same base network with only these two switches changed.



The pipeline. Switch 1 selects the input representation, and switch 2 selects the mixer. Their four combinations define the models compared below. Part 2 built switch 1, Part 3 built switch 2, and this part compares every setting.

The four models

Input representation	Attention mixing	Delay mixing
Raw	Raw Attention	Raw delay mixer
Decomposed	Attention + decomposition	Autoformer-inspired

Outputs and response

Notebook Outputs: 4.1 four-model validation comparison and contrasting forecast cases; 4.2 horizon errors, period-routed ridge included; 4.3 selected models and final results.

Write: Response 4.

Response 4. Complete these three steps:

- Before running Output 4.1, predict in at most two sentences which upgrade should help most on contexts where the slow component is especially prominent, and why.
- From Outputs 4.1 and 4.2, report the validation RMSEs and MSEs for the four neural models and the linear baselines on the established stations; compare the two architectural switches in both settings, and summarize how errors change across the forecast horizon.
- Based on validation performance, parameter count, and fitting time, identify the best deployment model for the established stations and for held-out Station 4, and justify each choice.

Reading for the curious (nothing here is assessed). The delay mixer and decomposition stream you built follow Autoformer, Wu et al. (2021), §§3.1–3.2, which also uses progressive moving-average decomposition and FFT-based autocorrelation with time-delay aggregation. The paper differs from this assignment in several ways: it uses an encoder–decoder rather than an encoder with a flattened forecast head, accumulates trend contributions through the decoder instead of using a separate input-trend head, and selects delays shared across a training batch rather than per example, without a fixed delay band.

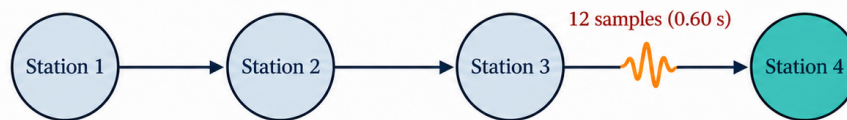
Looking Ahead — Why Temporal Context Is Not Always Enough (ungraded)

Every model above receives only one station's 96-sample context. That information boundary now becomes the forecasting bottleneck.

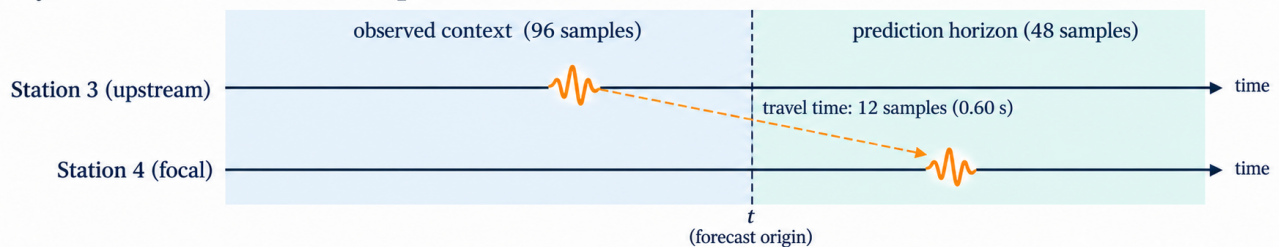
For this teaser only, consider a possible disturbance such as a loose part or a jam. It travels down the physical line from station 3 to station 4 in a known 12 samples (0.60 seconds). In the event case, station 3 has already recorded the warning during its context; station 4's context is still undisturbed, and the event first appears inside station 4's forecast horizon.

Now imagine two cases with *identical* station-4 contexts: in one, no disturbance is approaching; in the other, station 3's synchronized context already reveals an incoming event. A forecaster that sees only station 4's local history cannot tell these cases apart – it must return the same forecast for both, even though their true futures differ. No amount of additional temporal capacity fixes this: the information that would distinguish the two futures simply is not in the input. Only the synchronized station-3 context can distinguish them, because it reveals which future is approaching before station 4 ever records the event. The next assignment develops models that combine station relationships with temporal histories, allowing forecasts to depend on both *when* a pattern occurred and *where* it is moving.

Linear production line and approaching disturbance



Synchronized contexts and prediction



Same Station 4 history. Different future. Station 3 reveals what is approaching.

Task 2 — Leaderboard Challenge

2.1 Overview

You are given a real-world univariate time series that has been anonymised: its original identity, units and calendar dates have been removed. The series is sampled on a *strictly regular* grid.

Your task is to forecast the next **168 consecutive** observations beyond the end of the data you are given, using an **Autoformer** model. Your forecasts are evaluated against a held-out continuation of the series that you never see, on a public leaderboard.

You are also given a file of *optional external factors* measured alongside the target. Determine whether it helps your final prediction (Section 2.3).

Ranking is not by RMSE alone: model size and training length carry a penalty, so a smaller, cheaper model that matches a larger one will outrank it (Section 2.7).

Beyond the constraints in Section 2.4, the design of the pipeline — preprocessing, window construction, validation strategy, hyperparameters, training schedule, ensembling — is yours to decide and defend.

2.2 Data

You are given three files.

File	Contents
<code>student_train.csv</code>	<code>time_idx</code> , <code>value</code> — the observed history, 43,656 rows.
<code>student_test.csv</code>	<code>time_idx</code> only — the values are left empty for you to forecast.
<code>optional_external_data.csv</code>	<code>time_idx</code> , <code>feature_A</code> ... <code>feature_J</code> — optional variables, 43,824 rows (Section 2.3).

In `student_train.csv`, `time_idx` is an integer position $1, 2, \dots, 43,656$ and `value` is the observed measurement at that position.

Properties you may rely on

- The complete series contains 43,824 observations. You are given the first 43,656; the final 168 are held out for scoring.
- Sampling is uniform. For every t , $\text{time_idx}(t+1) - \text{time_idx}(t) = 1$ corresponds to exactly one real sampling interval. There are *no* irregular gaps of the kind found in market data. The length of that interval is not disclosed; recovering the periodic structure is part of the exercise.
- `student_train.csv` is complete: it contains no missing values. Missing observations in the original history, if any, were interpolated before release. The test set contains no interpolated values.
- The target is a non-negative physical quantity measured at a single location. It is strongly autocorrelated at short lags but that correlation decays quickly: the present value says a great deal about the next few steps and progressively less thereafter.
- The series is *not* dominated by a smooth long-run trend, and no single periodic cycle explains most of its variance. It is heavy-tailed: the level can change by an order of magnitude within the span of one forecast horizon. Expect errors that are large in absolute terms compared with a smoothly trending series.

- The optional variables are genuine measurements recorded on the same clock as the target — they are not derived from it. Six are continuous; four are binary indicators that are mutually exclusive at each time step.

2.3 The external data

`optional_external_data.csv` spans all 43,824 time steps. It therefore covers the 168 hidden positions as well as the history.

This is deliberate, and using it is permitted

Optional values over the forecast horizon; even though they are optional to use, are given to you on purpose. Using them is **not** leakage and **not** a violation of Section 2.4: they are external measurements, not the target. This mirrors a common practical setting in which forward-looking optional information is available at forecast time while the quantity of interest is not.

Whether it helps at all, which of the ten variables carry signal, and — in particular — *which portions* of it are worth feeding to your model are questions for you to settle experimentally on your own validation split. Consider carefully when each part of this file becomes available relative to the quantity being predicted, and where in your architecture each part should enter. Report what you find either way. A well-evidenced negative result is always a better option; an unsupported claim in either direction does not.

2.4 Requirements

These are the only hard constraints. Everything else is a design decision you are free to make.

1. **The architecture must be Autoformer.** Your forecasting model must be an Autoformer, and it must genuinely contain the two mechanisms that distinguish it from a vanilla Transformer:
 - (a) **Series Decomposition** — progressive separation of the signal into trend-cyclical and seasonal components; and
 - (b) **Auto-Correlation** — dependency discovery and aggregation at the level of *periods* via time-delay similarity, rather than point-wise dot-product attention.

Substituting a different sequence model — vanilla Transformer, Informer, LSTM, a gradient-boosted regressor — as your *submitted* model is not acceptable. You may of course build whatever you like alongside it for your own comparison.

2. **Forecast horizon is exactly 168 steps**, using `pred_len = 168`. Forecast indices run from `time_idx = 43657` through `time_idx = 43824`.
3. **Report over multiple random seeds.** Single-seed comparisons on this series are not trustworthy: the difference between two designs can be smaller than the spread across seeds. Any claim that one configuration beats another must rest on more than one run.
4. **Include an ablation** in your report showing what the optional file did for you — with and without it, across seeds, on your own chronological validation split.

5. You may write Autoformer from scratch or adapt a public reference implementation, **provided you cite what you adapted** or the one you developed in Question 1, and can explain every component you use.

2.5 Leaderboard submission

Submit your forecasts at:

<https://ai-651-leaderboard.vercel.app/>

There is no file upload and no CSV. The leaderboard provides a **predictions** field into which you paste your 168 forecast values **in chronological order** — that is, the forecast for `time_idx = 43657` first, through `time_idx = 43824` last:

```
v1, v2, v3, ... (168 comma-separated values) ..., v167, v168
~ ~
forecast for time_idx = 43657 time_idx = 43824
```

Enter exactly 168 numeric values. Generate the list programmatically and paste it — do not transcribe it by hand. Order matters: the leaderboard has no way to detect a reversed or rotated sequence, and will simply score it as a very poor forecast.

Alongside the forecasts you must declare your **total trainable parameters** and **number of training epochs**: both enter the ranking score defined in Section 2.7.

Two rules that should shape your whole strategy

1. You have at most **5 submissions** in total.

What this implies

The leaderboard is *not* a validation set, and you cannot treat it as a tuning signal. With 5 attempts and last-submission-wins scoring, selecting a model by leaderboard feedback is strategically dangerous.

Your own chronological validation is the best estimate of held-out performance that you have. Build a validation protocol you trust, make your model decisions against it, and treat each leaderboard submission as a confirmation rather than an experiment. Think carefully about how much evidence a single 168-step score really carries before you let it overrule a validation estimate built from many such blocks — and about how you would tell a genuine improvement from noise.

2.6 Metrics

With y_i the actual value and $\hat{y}_i$ the forecast over n points:

$$\begin{aligned} \text{MAE} &= \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, & \text{RMSE} &= \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \\ \text{sMAPE} &= \frac{100}{n} \sum_{i=1}^n \frac{2|\hat{y}_i - y_i|}{|y_i| + |\hat{y}_i|}. \end{aligned}$$

You are expected to track all three on your own validation split; they disagree in informative ways, and understanding where they disagree is part of the exercise. Ranking, however, uses RMSE together with two efficiency penalties, as follows.

2.7 Leaderboard score

Submissions are ranked by a single number, and **lower is better**:

$$\text{Score} = \text{RMSE} + \alpha \cdot P + \beta \cdot E$$

Term	Meaning
RMSE	root-mean-square error of your 168 forecasts against the held-out block
P	total number of <i>trainable</i> parameters in your final model
E	number of training epochs used to produce your submitted forecasts
α, β	You should not be interested in it :)

Accuracy is still the dominant term, but it is no longer the only one. Two submissions with the **same RMSE** are separated by cost: the one with fewer parameters and fewer epochs ranks higher. A larger model must therefore *earn* its size by producing enough extra accuracy to pay for it.

Declaring P and E

You must declare P and E with each submission.

- P is the total trainable parameter count of the model that produced your forecasts, as reported by for example `sum(p.numel() for p in model.parameters() if p.requires_grad)`.
- If you ensemble k models, P is the *sum* over all k and E the *total* epochs across all of them. Ensembling is allowed, but you pay for it.
- E counts the epochs actually spent producing the submitted forecasts. If you early-stopped at epoch e and then refit on the full series, count both.
- Both figures must be reproducible from your repository. Declared values that contradict your code will be treated as a misreported result.

2.8 Things to Report

While the problem is intentionally designed to be open-ended, your report should clearly describe the key decisions made throughout your solution. In particular, you should discuss your data processing and feature engineering steps, as well as the design choices behind your model architecture. For each major decision, provide the underlying intuition and reasoning that motivated your approach.

2.9 Questions to reflect

Support your reasoning with your own experiments where that is feasible.

- Q2.1** Auto-Correlation aggregates information across *lags* that score highly by series-wise correlation, whereas self-attention aggregates across *positions* that score highly by pairwise similarity. Characterise a class of signals for which the former is strictly the better inductive bias, and a class for which it discards something attention would have kept. What happens to each when the dominant period drifts over time?
- Q2.2** Autoformer’s decomposition extracts trend with a fixed-kernel moving average, applied repeatedly inside the network. What does that operator implicitly assume about the trend? Consider what it does at the boundaries of a window, to curvature, and to an abrupt level shift. Propose an alternative decomposition, state the inductive bias you are buying, and say what you would give up.
- Q2.3** Classical pipelines decompose once, as preprocessing. Autoformer interleaves decomposition into every block. Argue what this buys that one-shot decomposition cannot — and construct a case where the repeated version could actively hurt.
- Q2.4** The number of aggregated delays is set as $k = c \log L$ for input length L . Interpret this choice: what is the trade-off as k grows, and as it shrinks? Is the $\log L$ form principled or convenient? How should k interact with the number of distinct periodicities actually present in a signal?
- Q2.5** A horizon can be produced in one shot or by feeding predictions back as inputs. Compare the two on error accumulation, exposure bias, and the correlation structure of the resulting errors. Under what noise or dynamics would each be preferable, and how does the answer change as the horizon grows?
- Q2.6** Window-relative normalisation schemes remove each input window’s location and scale before modelling and restore them afterwards. What information is destroyed by doing so, and when does that information actually matter for the forecast? Describe a series where this helps a great deal and one where it provably loses something essential.
- Q2.7** optional variables may be known only up to the forecast origin, or known across the horizon as well. Does that distinction change where such variables should enter an encoder–decoder forecaster, or only how much they help? Describe a covariate that would be informative in the second regime and useless in the first, and one that is equally useful in both.
- Q2.8** Here the timestamps are replaced by an integer index, so no calendar features are available. What can a model no longer exploit? How might it recover periodic structure without being told the period — and what would you have to assume for that recovery to be trustworthy?
- Q2.9** Structural time-series methods (ETS, SARIMA, state-space models) encode strong assumptions and estimate few parameters. A decomposition Transformer assumes less and estimates far more. Where is the crossover — in series length, signal-to-noise ratio, and regularity — and how would you decide, before training, which side of it you are on?

Submission Guidelines

- **What to submit:** one PDF report covering both tasks; the report’s LaTeX source; the executed full-preset Task 1 notebook; and each numbered PDF figure the report includes.
- **Report Preparation:** The responses in Task 1 (Total:5) are compulsory to answer, while the Q2.1–Q2.9 reflection questions in Task 2 are not compulsory in the report, but they connect to concepts that may appear in quizzes or exams and are still important.
A good, structured report is always one that is brief but explanatory on its own. We do not prefer long paragraphs, AI-generated content, etc. Instead, each point should be supported by clear reasoning and proper justification, with relevant evidence wherever applicable for example, through plots, and figures.
- **Attribution:** you may use public deep-learning libraries and public reference code, **provided you cite it clearly** and can explain it. Uncited copying is plagiarism, and code or forecast values must not be shared with other students.
- The repository and generative-AI requirements are stated in the Instructions on page 1, and apply in full.

Before you submit Task 1

Check that every notebook check passes, that Outputs 1.1–4.3 are present, that the two deployment choices were written before Output 4.3 was run, that every experimental claim names its population and split, and that the Response 4 interpretation is at most 200 words.

Appendix: generator and fitting reference

None of these settings is needed to answer a question; they are collected here for completeness.

Anchoring and scaling, in full

Let $a = y_i(t - 1)$ be the last observed value and c one standard deviation of consecutive differences of established-station training readings. Then

$$x_j = \frac{y_i(t - L + j) - a}{c}, \quad j = 0, \dots, L - 1, \quad z_h = \frac{y_i(t + h) - a}{c}, \quad h = 0, \dots, H - 1, \quad (7)$$

where $\mathbf{x} \in \mathbb{R}^L$ is the anchored, scaled context received by every model and $\mathbf{z} \in \mathbb{R}^H$ is the corresponding target. Convert a normalized forecast back to reading units by multiplying by c and restoring the anchor: $\hat{y}_i(t + h) = a + c \hat{z}_h$.

Ridge closed form

The shared, sensor-specific, and period-routed ridge forecasters in Part 1 all fit the same closed-form least-squares map,

$$\hat{\mathbf{z}} = \mathbf{x}^\top \widehat{\mathbf{W}}, \quad \widehat{\mathbf{W}} = \left(X^\top X / n + \lambda I \right)^{-1} X^\top Z / n \in \mathbb{R}^{L \times H}, \quad (8)$$

where $X \in \mathbb{R}^{n \times L}$ stacks the n anchored training contexts as rows, $Z \in \mathbb{R}^{n \times H}$ stacks their targets, I is the $L \times L$ identity, and λ is a small positive constant that keeps the inverse well conditioned. This is one solve and requires no optimizer; only the population of rows differs among the three ridge forecasters.

Delay-mixer hyperparameters, in full

Choice	Why
delays restricted to $\{8, \dots, 48\}$	below 8 the score mostly measures the sample-to-sample smoothness of the signal; above 48 a delay covers more than half a 96-sample ring
keep the $K = \lceil 2 \ln L \rceil$ best	a small, length-dependent budget; at $L = 96$ this is $K = 10$
a learned temperature $\vartheta > 0$	it controls how sharply the surviving delays are separated: small ϑ concentrates nearly all weight on the best delay, large ϑ spreads it almost evenly
Quantity	Value
stations, samples	4 stations, 15,360 samples each, sampled at 20 Hz
product schedule	Product A, B, C repeat globally every 192 samples
actual primary period	one fixed value per operating run, shared across stations: Product A 14–18, Product B 21–27, and Product C 28–36 samples; a deterministic low-discrepancy schedule spreads repeated runs through each range
phase	$\phi_i(t+1) = \phi_i(t) + 2\pi/P^{(r)}$, accumulated across boundaries
repeating part	$s_i(t) = A_i \sin \phi_i(t)$; fixed station amplitudes and mounting phases
slow part	$b_i(t) = \ell_i + B_i \sin(2\pi t/240 + \theta_i)$; fixed calibration offsets, drift magnitudes, and drift phases per station
noise	independent, standard deviation 0.09 m/s^2
ridge routing	period estimated from the context, then matched to one of the supplied 1.5-sample pace bands; one ridge matrix is fitted per populated band
ridge penalty	one small λ , fixed once on a chronological holdout inside the established-station training region; station 4 is excluded
spatial teaser	a separately constructed event travels from station 3 to station 4 in 12 samples

References

- Wu et al. (2021). Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting, §§3.1–3.2.
- Vaswani et al. (2017). Attention Is All You Need.
- Zeng et al. (2023). Are Transformers Effective for Time Series Forecasting?
- Hyndman and Athanasopoulos. Forecasting: Principles and Practice, §5.10.